

# Documentação Técnica do Datapath — Falcon-8 Pipeline

## Visão Geral

O Falcon-8 é um processador RISC de 8 bits com instruções de 16 bits, implementado no Logisim-Evolution com pipeline de 3 estágios. A arquitetura é inspirada no RISC-V, adaptada para 8 bits com conjunto de instruções próprio.

---

## Especificações Gerais

| Parâmetro              | Valor                      |
|------------------------|----------------------------|
| Largura dos dados      | 8 bits                     |
| Largura das instruções | 16 bits                    |
| Registradores          | 8 (R0–R7), R0 fixo em 0    |
| Endereçamento          | 8 bits (0x00–0xFF)         |
| Memória de instruções  | ROM 256×8 (word-addressed) |
| Memória de dados       | RAM 256×8                  |
| Clock                  | Borda de descida           |
| Estágios do pipeline   | 3                          |

---

## Formato das Instruções

O processador suporta 5 tipos de instrução, todos com 16 bits de largura:

### Tipo R — Operações entre registradores

[15:12] opcode (4 bits)  
[11:9] rd (3 bits) — registrador destino

[8:6] rs1 (3 bits) — operando 1  
[5:3] rs2 (3 bits) — operando 2  
[2:0] funct (3 bits) — função da ALU

Instruções: **ADD**, **SUB**, **AND**, **OR**, **SLT**, **SLL**, **SRL**

### Tipo I — Operações com imediato

[15:12] opcode (4 bits)  
[11:9] rd (3 bits)  
[8:6] rs1 (3 bits)  
[5:0] imm (6 bits, com sinal)

Instruções: **ADDI**, **LW**, **SLLI**

### Tipo S — Store

[15:12] opcode (4 bits)  
[11:9] rs2 (3 bits) — dado a gravar  
[8:6] rs1 (3 bits) — endereço base  
[5:0] offset (6 bits, com sinal)

Instrução: **SW**

### Tipo B — Branch condicional

[15:12] opcode (4 bits)  
[11:9] rs1 (3 bits)  
[8:6] rs2 (3 bits)  
[5:0] offset (6 bits, com sinal) — em palavras

Instruções: **BEQ**, **BNE**, **BLT**

### Tipo J — Jump

[15:12] opcode (4 bits)  
[11:9] rd (3 bits) — registrador de retorno  
[8:0] offset (9 bits)

Instrução: **JAL**

---

## Banco de Registradores

8 registradores de 8 bits (R0–R7). R0 é hardwired em 0 — qualquer escrita nele é ignorada e qualquer leitura retorna 0. Os demais são de uso geral.

Os campos **rs1**, **rs2** e **rd** da instrução endereçam diretamente os registradores com 3 bits cada.

---

## Estágios do Pipeline

### Estágio 1 — IF (Instruction Fetch)

O PC é um registrador de 8 bits que endereça a ROM. A cada ciclo, a instrução de 16 bits no endereço atual é lida e o PC é incrementado em 2.

O endereço do próximo PC (**nextPC**) pode vir de três fontes, selecionadas por um MUX:

- **PC + 2** — fluxo normal
- **PCbranch** — endereço calculado para branch (PC + offset)
- Endereço de jump (**JAL**) — calculado a partir do imediato de 9 bits

Entre os estágios IF e ID existe um registrador de pipeline que armazena a instrução (**instrucao**) e o valor **PC+2** (endereço de retorno para JAL).

**Flush:** quando um branch é tomado ou um jump ocorre, o sinal **FLUSHD** força o registrador IF/ID a receber um NOP (instrução zerada), descartando a instrução que entrou no pipeline incorretamente.

---

### Estágio 2 — ID/EX (Decode + Execute)

É o estágio mais denso do processador. Ele realiza quatro tarefas em paralelo no mesmo ciclo:

**Decodificação da instrução:** O campo **opcode** (bits [15:12]) é separado e enviado à Unidade de Controle. Os campos **rs1**, **rs2**, **rd**, **funct** e os imediatos são extraídos por splitters.

**Geração do imediato (ImmGen):** O ImmGen recebe o tipo de imediato via **ImmSel** (3 bits) e entrega um valor de 8 bits com extensão de sinal para a ALU. Os tipos suportados são: imediato I (6 bits), imediato S (6 bits), imediato B (offset de branch), imediato J (offset de jump de 9 bits).

**Leitura dos registradores:** O banco de registradores é lido com **rs1** e **rs2**, produzindo **rd1** e **rd2**. Antes de entrarem na ALU, esses valores passam por MUXes de forwarding

que verificam se a instrução anterior escreveu no mesmo registrador — se sim, o valor mais recente é usado diretamente, sem esperar o writeback.

**Execução na ALU:** A ALU recebe dois operandos (um deles pode ser o imediato, dependendo do sinal `ALUsource`) e realiza a operação indicada por `ALUoperation` (3 bits). O resultado é `ALUdata` e um flag de zero (`ALUzero`) é gerado para uso nos branches.

**Controle de branch/jump:** `ALUzero` combinado com o sinal `Branch` da UC determina se um branch é tomado. O endereço alvo (`PCbranch`) é calculado como  $PC + offset \times 2$ . Para JAL, o sinal `Jump` direciona o PC para o endereço do jump e o `rd` recebe `PC+2`.

Entre os estágios ID/EX e MEM/WB existe um registrador de pipeline que propaga: resultado da ALU (`ALUdata`), dado para store (`rdstore`), sinais de controle de memória (`MemRead`, `MemWrite`, `MemToReg`), sinal de escrita em registrador (`RegWrite`), endereço de destino (`rd`), e informações de forwarding (`rdP`, `RegWriteP`).

---

## Estágio 3 — MEM/WB (Memory + Writeback)

Acesso à memória RAM e escrita no banco de registradores acontecem no mesmo estágio.

**Acesso à memória:**

- `LW`: RAM lê no endereço calculado pela ALU e entrega `MemData`
- `SW`: RAM escreve `rdstoreR` no endereço calculado, habilitado por `MemWriteR`
- `MemRead` e `MemWrite` controlam qual operação ocorre

**Writeback:** O MUX `MemToReg` seleciona o que será gravado no banco de registradores:

- `0` → resultado da ALU (`dataR`) — instruções aritméticas
- `1` → dado lido da memória (`MemData`) — instrução `LW`

A escrita ocorre no registrador endereçado por `rdP`, habilitada por `RegWriteP`.

---

## Sinais de Controle

A Unidade de Controle decodifica o `opcode` e gera os seguintes sinais:

| Sinal                     | Largura | Descrição                   |
|---------------------------|---------|-----------------------------|
| <code>ALUoperation</code> | 3 bits  | Seleciona a operação da ALU |

|           |        |                                       |
|-----------|--------|---------------------------------------|
| ALUSource | 1 bit  | 0 = registrador, 1 = imediato         |
| ImmSel    | 3 bits | Seleciona o formato do imediato       |
| Branch    | 1 bit  | Habilita desvio condicional           |
| Jump      | 1 bit  | Habilita jump incondicional (JAL)     |
| MemRead   | 1 bit  | Habilita leitura da RAM               |
| MemWrite  | 1 bit  | Habilita escrita na RAM               |
| MemToReg  | 1 bit  | Seleciona fonte do writeback          |
| RegWrite  | 1 bit  | Habilita escrita no banco             |
| Store     | 1 bit  | Indica instrução de store             |
| FLUSH     | 1 bit  | Limpa o pipeline (branch/jump tomado) |

---

## Tratamento de Hazards

### Hazard de dados — RAW (Read After Write)

Resolvido por **forwarding**: os sinais **rdP** e **RegWriteP** propagam junto com o resultado da ALU até o estágio seguinte. MUXes no início da ALU comparam **rdP** com **rs1** e **rs2** da instrução atual e, se forem iguais e **RegWriteP** estiver ativo, substituem o valor do banco pelo resultado já calculado.

### Hazard de dados — Load-Use

Em pipelines de 5 estágios, o LW seguido imediatamente de uma instrução que usa o resultado exige um ciclo de stall. No Falcon-8 com 3 estágios isso não ocorre: quando a instrução seguinte ao LW chega no ID/EX para ler os registradores, o LW já passou pelo MEM/WB e gravou o valor. **Nenhum stall é necessário.**

### Hazard de controle — Branch e Jump

Resolvido por **flush**: quando um branch é tomado ou um JAL ocorre, o sinal **FLUSHD** injeta um NOP no registrador IF/ID, descartando a instrução que entrou no pipeline no ciclo seguinte. Apenas 1 ciclo de penalidade.

### WAR e WAW

Não ocorrem nesta arquitetura de 3 estágios. A ordem de leitura e escrita é preservada por design — o estágio de decodificação sempre lê após o writeback da instrução anterior ter sido concluído.

---

## Fluxo Completo de uma Instrução

- Ciclo N: IF → busca instrução na ROM, PC+1
- Ciclo N+1: ID/EX → decodifica, lê regs, executa ALU
- Ciclo N+2: MEM/WB → acessa RAM (se LW/SW), grava resultado

Enquanto a instrução A está no MEM/WB, a instrução B está no ID/EX e a instrução C está no IF — três instruções em execução simultânea a cada ciclo.

---

## Mapa de Memória (Módulos SFI e CVA)

| Endereço | Nome              | Tipo    |
|----------|-------------------|---------|
| 0x10     | DIST_ATUAL        | Entrada |
| 0x11     | VEL_ATUAL         | Entrada |
| 0x12     | DIST_SEGURA       | Entrada |
| 0x13     | VEL_MAX_PERMITIDA | Entrada |
| 0x14     | VEL_FRENTE        | Entrada |
| 0x15     | DIST_FRENTE       | Entrada |
| 0x20     | CMD_FREIO         | Saída   |
| 0x21     | STATUS_SISTEMA    | Saída   |
| 0x22     | CMD_ACCELERADOR   | Saída   |
| 0x23     | VEL_ALVO          | Saída   |